

Assignment 2 – George Boufis – Report

The project was implemented using **Neo4j AuraDB Free** and the **Cypher query language**.

Initially, I designed a knowledge graph to represent relationships between: **Owners, Apartments, Roommates, Amenities, Rules**.

Each entity was created as a **node label** (**:Owner, :Apartment, :Roommate, :Amenity, :Rule**) and connected through **relationships** such as: (:Owner) - [:OWNS] -> (:Apartment) , (:Apartment) - [:HAS_RULE] -> (:Rule), (:Apartment) - [:HAS_AMENITY] -> (:Amenity), (:Roommate) - [:LIVES_IN] -> (:Apartment).

Dummy data were inserted manually via Cypher MERGE statements to populate the graph with realistic values (e.g., locations, rents, lifestyles, and rules).

I chose Neo4j because it provides **visual representation of relationships**, which helps in exploring how owners, apartments, and roommates are connected.

Attributes like **rent, location, number of bathrooms**, and **hasView** were modeled as properties of the Apartment node.

Constraints such as **rules** (“Smoking is forbidden”, “No pets allowed”) were represented as connected Rule nodes rather than as properties, allowing easier query flexibility.

Each Amenity (e.g., Wi-Fi, Parking, Heating) was modeled as a separate node type, to enable querying apartments by shared features.

I used **MERGE** instead of CREATE to avoid data duplication when re-running the script.

I used ChatGPT for guidance on:

Structuring the Cypher schema, debugging missing relationships, translating the competency questions into Cypher syntax.

#	Competency Question	Cypher Query
1	Which apartments have rent below 400 euros?	<pre>MATCH (a:Apartment) WHERE a.rent < 400 RETURN a.apartmentId, a.rent;</pre>
2	Which apartments have Wi-Fi?	<pre>MATCH (a:Apartment)-[:HAS_AMENITY]->(:Amenity {name:"Wi-Fi"}) RETURN a.apartmentId;</pre>
3	Which apartments have two bathrooms?	<pre>MATCH (a:Apartment) WHERE a.numberOfBathrooms = 2 RETURN a.apartmentId;</pre>
4	Which apartments are located in the city center?	<pre>MATCH (a:Apartment) WHERE a.location = "City Center" RETURN a.apartmentId, a.location;</pre>
5	In which apartments does the owner forbid smoking?	<pre>MATCH (o:Owner)-[:OWNS]->(a:Apartment)-[:HAS_RULE]->(:Rule {description:"Smoking is forbidden"}) RETURN o.name, a.apartmentId;</pre>
6	How many apartments have parking?	<pre>MATCH (a:Apartment)-[:HAS_AMENITY]->(:Amenity {name:"Parking"}) RETURN COUNT(a) AS NumberOfApartments;</pre>
7	How many apartments have more than one bathroom?	<pre>MATCH (a:Apartment) WHERE a.numberOfBathrooms > 1 RETURN COUNT(a);</pre>
8	How many apartments does each owner have for rent?	<pre>MATCH (o:Owner)-[:OWNS]->(a:Apartment) RETURN o.name, COUNT(a) AS NumberOfApartments;</pre>

The graph successfully models a small rental property domain, allowing flexible queries on ownership, apartment characteristics, and tenant rules.

Cypher queries were able to answer all competency questions directly through graph relationships, demonstrating the expressiveness of graph databases for semantic data exploration.